Space Robotics – Assignment 3

Lucas Plumbohm
24949202

Elijah Spannenberg
24767473

Mathis Beissner
26665176

1 Generative AI Disclosure

Generative AI tools were used to assist with analysing error messages during the debugging process, creating helper scripts for advanced visual debugging, formatting this report as a LaTeX document, and correcting spelling and wording throughout the report. OpenAI Codex also assisted with drafting Advanced 1, 2 and 4 from the implementation and saved experiment results.

2 Statement of Individual Contributions

Task	Completed?	Team member	Contribution
Perception 1	Yes	Mathis	All
Perception 2	Yes	Mathis	All
Perception 3	Yes	Mathis	All
Planning 1	Yes	Lucas	All
Planning 2	Yes	Lucas	All
Planning 3	Yes	Lucas	All
Advanced 1	Yes	Elijah	All
Advanced 2	Yes	Elijah	All
Advanced 4	Yes	Elijah	All
Report writing			
Recorded videos			

3 Project Overview

TBA

4 Implementation

TBA

4.1 Perception 1: Collect an Image Dataset of the Artefacts

The goal of this task was to create an image dataset for artefact detection. A subscriber was added to the `CaveExplorer` node so that image capture could be requested by manually publishing a `std_msgs/msg/Empty` message. The callback below sets a flag, prompting the next image callback to save the images and reset the flag. A new depth subscriber was synchronised with the existing RGB subscriber so that each matching image pair is processed by a single callback.

```
    def capture_callback(self, msg):
        self.capture_requested_ = True
        self.get_logger().info('Requesting image capture.')
```

Listing 1: Implementation of capture callback

To investigate how many images were needed, test models were trained on subsets containing 10, 20 and 30 images of the alien artefact only. Performance improved with dataset size, with the 30-image dataset producing the best results among the tested sizes. Precision and recall both reached 1.0 on the test split (Figure 1). Based on this result, at least 30 images were collected per artefact class. The model and training procedure are described in more detail in Perception 2.

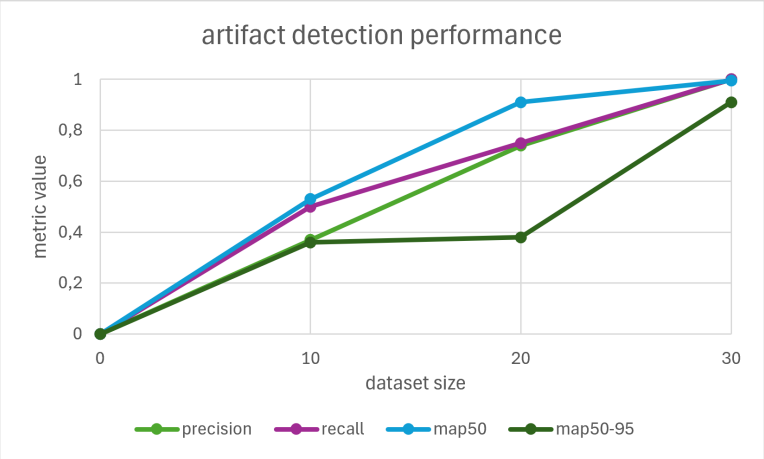


Figure 1: Detection performance for the alien artefact with increasing dataset size. Precision and recall reach 1.0 on the test split at 30 images.

The final dataset contains 203 images, including scenes with multiple artefacts and a few negative examples without artefacts to help reduce false positives. Some images contain multiple instances of a class, resulting in

more than 30 instances per class. Figure 2 shows a corresponding RGB and depth image pair containing the gem and mushroom artefacts.

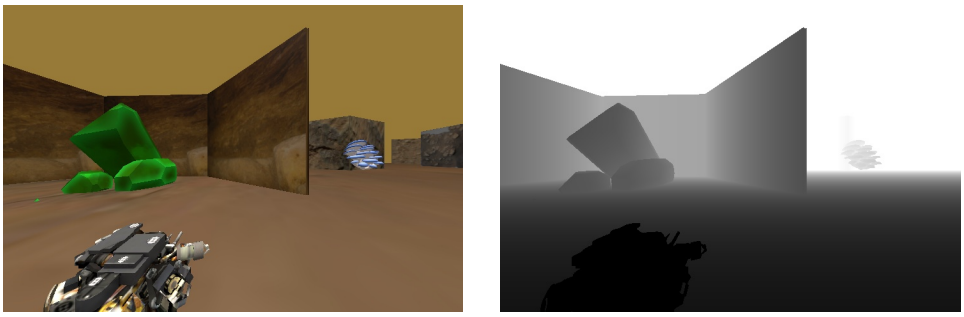


Figure 2: Example RGB image (left) and corresponding depth image (right) from the collected dataset, showing the gem and mushroom artefacts.

4.2 Perception 2: Detect Artefacts with Computer Vision

The goal of this task was to train and integrate an artefact detector using the dataset collected in Perception 1. The Ultralytics YOLO framework was used with a pretrained YOLO26 nano model to keep inference time and computational requirements low. The data was divided into training, validation and test sets for updating model weights, monitoring training and evaluating the final model, respectively. Several hyperparameter combinations were tested before selecting a learning rate of 0.002, a batch size of 4 and the AdamW optimiser.

Separate models were trained for RGB and depth images. For depth detection, distances from 0 to 15 metres were mapped to greyscale values to improve the visual contrast of the artefacts. Annotations beyond this range were removed. Stop-sign annotations also required removal because the sign is a texture on a flat surface and cannot be distinguished in depth images.

Table 1 summarises the saved test results for the final RGB and depth models. The depth model achieved a precision of 1.0 and a higher mAP@50–95, while the RGB model achieved slightly higher recall and mAP@50.

Metric	RGB model	Depth model
Precision	0.985	1.000
Recall	0.981	0.975
mAP@50	0.977	0.971
mAP@50–95	0.796	0.840

Table 1: Test-set detection metrics from the saved model evaluations.

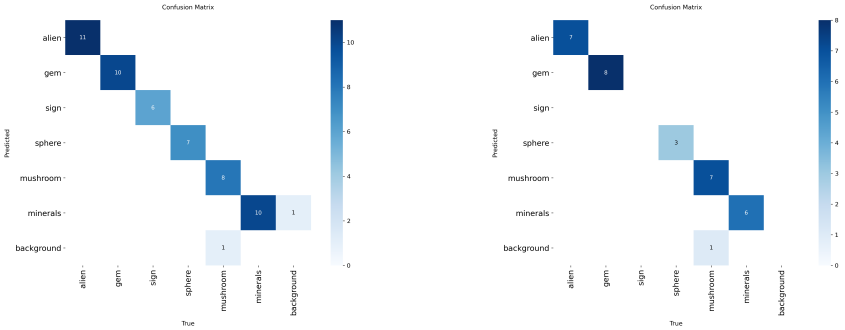


Figure 3: Test-set confusion matrices for the RGB model (left) and the depth model (right). Both models miss one mushroom; the RGB model also produces one false-positive mineral detection. The depth model produces no false positives, and its stop-sign row and column are empty because this class was excluded from the depth labels.

Integrating the detection into ROS proved difficult. Initially, the detection was implemented to run directly inside `image_callback()` in the `CaveExplorer` node. Processing took longer than the 0.2-second image interval, blocking the callback and causing frames to be missed. Inference was therefore moved to a separate node using the `ros_yolo` package. The image callback publishes the selected RGB or depth image to this node, and returned detections trigger `artifact_detections_callback`. This callback retrieves the corresponding image and robot pose at capture time, then coordinates image annotation and artefact localisation, as described in Perception 3. Colour-coded bounding boxes and class labels are published with the images for display in RViz.

Both RGB and depth detections where implemented and the input modality is selected using the `use_depth_detection` parameter in `detection_config.yaml`. Since detection still affected runtime performance, `detection_every_n_images` was added to reduce processing frequency by skipping images between detections. During a cave traversal with a detection confidence threshold of 0.7, no false positives were observed with either model.

Figure 4 illustrates the differences in detection performance between the inputs. Both models detect the nearby mushroom with high confidence, while only RGB detects the stop sign and distant sphere. The stop sign has no distinct depth appearance, and the sphere lies beyond the selected depth range. During exploration, depth detection nevertheless identified all artefacts correctly when they were within range.

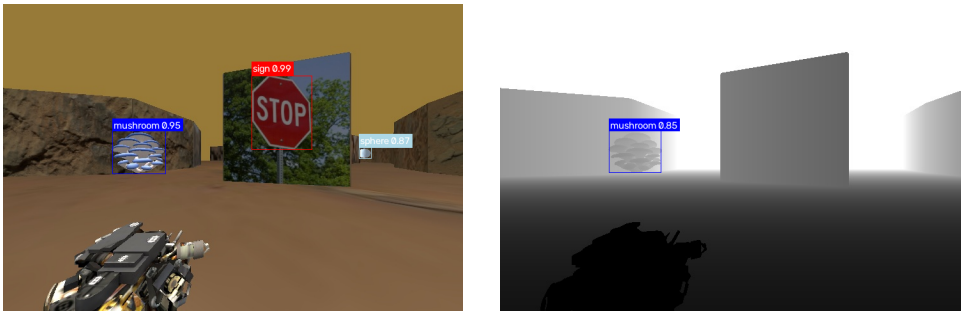


Figure 4: Artefact detections using RGB (left) and depth (right). Both models detect the mushroom; the stop sign and distant sphere are detected only in the RGB image.

4.3 Perception 3: Artefact Localisation and Display

The goal of this task was to estimate the map positions of detected artefacts and display them in RViz. The horizontal centre of each bounding box was used to calculate the artefact's bearing from the camera's field of view and focal length. To calculate the artefact's distance from the camera, first the depth value at the bounding-box centre was retrieved from the depth image. This depth value was then divided by the cosine of this bearing to obtain the distance. Combining this distance and artefact angle with the robot's pose gave the artefact's position in the world frame.

Initially, localisation used the robot's pose at the time the detection result was processed. Since model inference introduced a delay, this produced incorrect positions, particularly while the robot was turning. To correct this, `get_pose_2d()` was extended to accept the image timestamp and retrieve the corresponding pose. TF lookups then sometimes failed with "Lookup would require extrapolation into the future", indicating that the required transform was not yet available. Making the detection callback and pose lookup asynchronous allowed the system to wait for the transform.

```python
def localise_artifact(self, robot_pose, depth_image, detection):

    # Camera parameters
    HORIZONTAL_AOV = 2/3 * math.pi
    IMAGE_WIDTH = 720
    FOCAL_LENGTH = IMAGE_WIDTH / (2 * math.tan(HORIZONTAL_AOV / 2))

    # Calculate artefact angle from detection coordinates
    box = detection.bbox
    cx, cy = round(box.center.position.x),
        round(box.center.position.y)
    bearing = math.atan2((IMAGE_WIDTH/2 - cx), FOCAL_LENGTH)
    absolute_angle = wrap_angle(robot_pose.theta + bearing)

    # Calculate the artifact distance from the depth image
    depth = float(depth_image[cy, cx])
    if not np.isfinite(depth) or depth <= 0:
        return
    distance = depth / math.cos(bearing)

    # Calculate the artifact's position in the world frame.
    point = Point()
    point.x = robot_pose.x + distance * math.cos(absolute_angle)
    point.y = robot_pose.y + distance * math.sin(absolute_angle)
    point.z = 1.0

    # Save the artifact's position to the list
    self.artifact_locations_.append(point)

    # Publish the markers
    self.publish_artifact_markers()
```

Listing 2: Artefact position estimation from a detection and depth image

To archieve usable artifact localizations, repeated observations of the same artefact needed to be combined into a single position estimate. This was achieved by using a Density-Based Spatial Clustering approach (DBSCAN) from the scikit-learn package. This algorithm groups nearby position estimates using a distance threshold and a minimum number of neighbouring points, required to form a core point of a cluster. Several distance thresholds were tested, with 3 metres producing the best grouping in the test scenario. All observations were assigned to the intended artefact clusters in that run, including the two separate chunks of the mineral artefact, which were correctly treated as one artefact. Figure 5 compares the markers before and after clustering.



Figure 5: Artefact localisation markers before clustering (left) and after DBSCAN clustering (right). Repeated observations are combined into a single estimated position for each artefact.

4.4 Planning 1: Autonomously Explore the Cave

The goal for planning task 1 was to autonomously explore the area. Frontier based exploration was used for this, by searching the `map_msg/OccupancyGrid` to find any free cells that were neighbouring unknown cells.

```
def is_frontier_cell(self, pos):
    """Returns if a cell is occupied on the map"""
    if self.occupancy_map[pos] != 0: #FREE space
        return False


    for neighbor in self.get_neighbours(pos):
        if self.occupancy_map[neighbor] == -1: #UNKOWN space
            return True
    return False
```

Listing 3: Frontier cell detection from occupany grid

These were then 'clustered' by checking if there were frontier cells neighbouring each other. These clusters were then returned in a larger array to later calculate the best cluster to approach.

```
def find_frontier_clusters(self):
    """Returns array of frontier clusters"""
    visited = np.zeros(self.occupancy_map.shape, dtype=bool)
    visited_neighbor = np.zeros(self.occupancy_map.shape,
        dtype=bool)
    frontiers = []
    starting_pose = (0,0)
    queue = deque([starting_pose])

    while queue:
        current_cell = queue.popleft()

        if self.is_frontier_cell(current_cell):
            cluster = [] #find all the frontier nodes in a cluster
            inner_queue = deque([current_cell])

            while inner_queue:
                frontier_cell = inner_queue.popleft()
                if visited[(frontier_cell)] == True:
                    continue
                visited[frontier_cell] = True
                cluster.append(frontier_cell)
                for neighbor in self.get_neighbours(frontier_cell):
                    if self.is_frontier_cell(neighbor):
                        inner_queue.append(neighbor)

            if len(cluster) >= 3: # only allow clusters of a certain
                size
                frontiers.append(cluster)

        visited[current_cell] = True
        for neighbor in self.get_neighbours(current_cell):
            if not visited_neighbor[neighbor]:
                visited_neighbor[neighbor] = True
                queue.append(neighbor)

    return frontiers
```

Listing 4: Frontier cell clustering

We used the following code snippit to find the cell indexes of neighbouring cells

```
def get_neighbours(self, pos): # PLANNER 1
    """Returns neighbors to a cell"""
    x,y = pos
    neighbors = []
    for dx in [-1, 0, 1]:
        for dy in [-1, 0, 1]:
            if dx == 0 and dy == 0:
                continue
            nx, ny = x + dx, y + dy
            width, height = self.occupancy_map.shape
            if 0 <= nx < width and 0 <= ny < height:
                neighbors.append((nx,ny))
    return neighbors
```

Listing 5: Find neighbouring cell indexes

Next we had to find the best frontier cluster to approach. We did this by calculating the average position of each cell in a cluster and choosing the nearest option to the robot. However, due to the fact the `map_msg/OccupancyGrid` held data in cells, we must convert these values into the relative world coordinates. This conversion can be seen in lines 16-19 of Listing 6

```python
def planner_nearest_frontier(self):
    """Go to the nearest frontier"""

    pose = self.get_pose_2d()
    frontiers = self.find_frontier_clusters()

    if frontiers: # if there are any unexplored frontiers
        best_frontier = None
        min_dist = 9999999 # A large number

        for clusters in frontiers:
            cluster_world_x = []
            cluster_world_y = []

            for frontier in clusters:
                cluster_world_x.append(self.xlim_[0]+
                                       frontier[0]*self.map_resolution)
                cluster_world_y.append(self.xlim_[0]+
                                       frontier[1]*self.map_resolution)

            centroid_x = np.mean(np.array(cluster_world_x), axis=0)
            centroid_y = np.mean(np.array(cluster_world_y), axis=0)
            centroid = (centroid_x,centroid_y) # average x, y
                coordiantes of cells in a cluster

            distance =
                np.linalg.norm(centroid-np.array((pose.x,pose.y)))

            if distance < min_dist:
                min_dist = distance
                best_frontier = (centroid[0],centroid[1])

        goal = Pose2D(x = best_frontier[0], y = best_frontier[1])
        self.planner_go_to_pose2d(goal)
```

Listing 6: Approach the nearest frontier cluster

4.5 Planning 2: Close-Range Inspection

This task was to approach any artifacts the robot found during its exploration. This was done by checking for artifact detections every main loop, and if found, going to the last seen artifact pose in `self.artifact_locations_` from the perception 3 task.

```
def planning_approach_artifact(self):
    """Go to last seen artifact"""
    robot_pose = self.get_pose_2d()
    artifact_point =
        self.artifact_locations_[len(self.artifact_locations_)-1] #
        Get the last seen artifact
    distance = np.linalg.norm((artifact_point.x,artifact_point.y) -
        (robot_pose.x,robot_pose.y)) - 1 # Near the artifact point
    heading = np.atan(((artifact_point.x - robot_pose.x) /
        (artifact_point.y - robot_pose.y))) # in Rads
    # self.get_logger().info(f'Heading (Degrees):
        {np.degrees(heading)}')
    x_increase = distance * np.cos(heading)
    y_increase = distance * np.sin(heading)
    goal = Pose2D(
        x = robot_pose.x + x_increase,
        y = robot_pose.y + y_increase,
        theta = heading
    )
    self.planner_go_to_pose2d(goal)
    self.visited_artifact_locations_.append(artifact_point)
```

Listing 7: Approach the last seen artifact

4.6 Planning 3: Behaviour Switching

The aim for this task was to avoid visiting the same artifact more than once. Our solution for this was quite simple, whenever the robot visits an artifact, the pose of that artifact is added to an array, as seen at the end of Listing 7, so whenever a new artifact is detected, it checks if the pose of that artifact is already in that array and if so it ignores that detection.

```
if self.artifact_found_ and
    self.artifact_locations_[len(self.artifact_locations_)-1] not
    in self.visited_artifact_locations_: # check if a new artifact
    is found
    self.planner_type_ = PlannerType.APPROACH_ARTIFACT
else:
    self.planner_type_ = PlannerType.NEAREST_FRONTIER
```

Listing 8: Approach the last seen artifact

4.7 Advanced 1: Robust Perception

The goal of this task was to extend artefact detection to poor lighting and motion blur. The RGB images were degraded before being sent to the detector, while the original depth measurements were retained for localisation. Darkness was simulated by multiplying pixel intensities by 0.7, 0.4 and 0.2 for mild, moderate and severe conditions. Horizontal motion blur averaged neighbouring pixels using kernel lengths of 5, 11 and 21 pixels. Combined conditions applied darkness followed by blur at the same severity. Neither operation changes image geometry, so the existing bounding-box annotations remain valid.

```
darkened = image.astype(np.float32) * brightness_factor
darkened = np.clip(np.rint(darkened), 0, 255).astype(np.uint8)
kernel = np.ones((1, int(blur_length)), dtype=np.float32) /
    blur_length
degraded = cv2.filter2D(darkened, -1, kernel,
                    borderType=cv2.BORDER_REFLECT_101)
```

Listing 9: Core darkness and horizontal motion-blur operations, with input checks omitted

The original split was preserved: 142 training, 30 validation and 31 test images. All degraded versions remained in their source split. An initial robust model used clean images, darkness and blur separately. The final model was fine-tuned from the original RGB checkpoint using a mixture of 40% clean images and 20% each of darkness, blur and combined degradation.

Training used AdamW, a learning rate of 0.0002, batch size 4 and image size 640, with a maximum of 100 epochs and early-stopping patience of 20.

Figure 6 shows the saved test comparison across all ten conditions. Severe darkness reduced the baseline mAP@50–95 to 0.652, and severe blur reduced it to 0.562. The final robust model achieved 0.825 and 0.755 respectively. The largest improvement was under combined severe degradation: mAP@50–95 increased from 0.073 to 0.762, compared with 0.676 for the initial robust model. Clean-image mAP@50–95 also increased from 0.813 to 0.837, although clean mAP@50 fell slightly from 0.995 to 0.994.

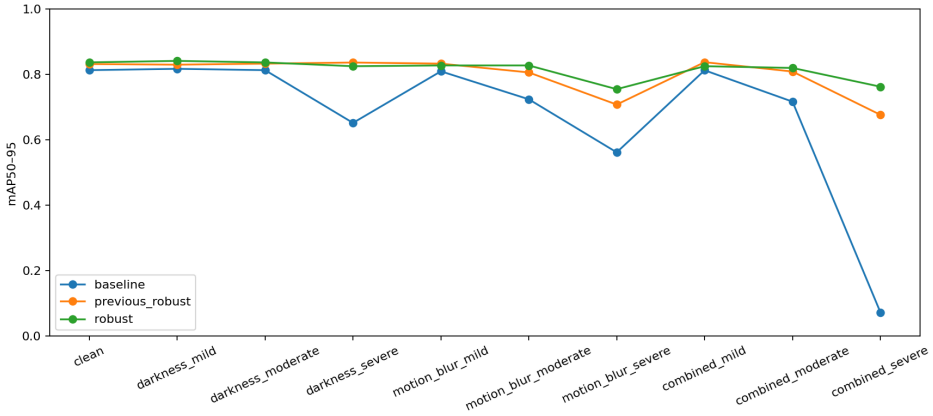


Figure 6: Saved test-set mAP@50–95 for the original detector (baseline), the first fine-tuned model (previous_robust), and the final model trained with combined degradation (robust). These are offline detection results.

For precision and recall, one confidence threshold per model was selected on mixed validation data and held fixed across test conditions: 0.172 for the baseline and 0.342 for the final model. Severe combined-condition recall increased from 0.000 to 0.960 at these operating points. AP was evaluated separately using predictions down to confidence 0.001; these results are a separate evaluation from the Perception 2 table.

The live pipeline publishes the original image, degraded input and annotated detections for RViz, retaining timestamps to associate detections with the correct depth image and robot pose. Figure 7 reserves space for this runtime evidence. The experiment demonstrates robustness to the tested synthetic effects, rather than physical dust or sensor noise. Further work would test independent capture sequences, measure localisation error, and include a clean-only fine-tuning control to separate the benefit of augmentation from additional training.

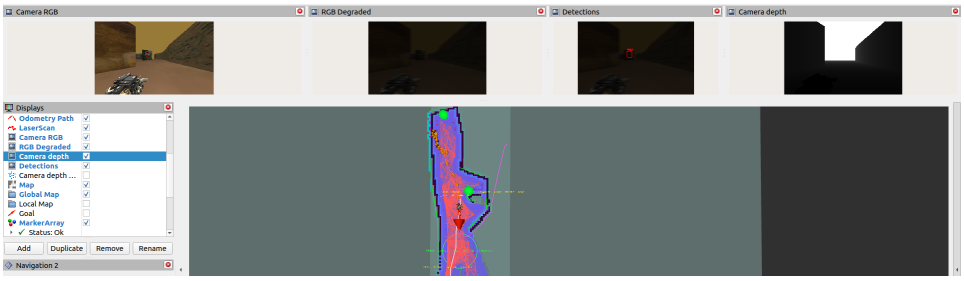


Figure 7: Runtime perception display: original image, degraded image, degraded detector output, and camera depth view.

4.8 Advanced 2: Cave Geometry Analysis

The goal of this task was to identify wide accessible areas and narrow passages as the cave map developed. A separate geometry node processes the latest occupancy grid and robot pose every two seconds. Occupancy values from 0 to 25 are treated as free; unknown cells, higher values and the map boundary block movement. A Euclidean distance transform gives each cell's distance to the nearest blocked cell centre. Subtracting half a cell diagonal conservatively accounts for the area occupied by that cell.

```
free = (grid >= 0) & (grid <= free_threshold)
padded = np.pad(free, 1, constant_values=False)
distances = ndimage.distance_transform_edt(padded)[1:-1, 1:-1] *
    resolution
clearance = np.maximum(0.0, distances - resolution / math.sqrt(2))
traversable = free & (clearance >= robot_radius + safety_margin)
```

Listing 10: Conservative clearance and traversability calculation

A four-connected flood fill from the robot identifies reachable cells, requiring clearance of at least the 0.5 m robot radius plus a 0.1 m safety margin. The maximum clearance in this component defines the widest accessible disc, displayed as a green circle with its diameter. This measures local open space rather than the total area of a chamber. Recomputing from the accumulated map allows the estimate to change after exploration or SLAM corrections. Unknown boundaries produce provisional measurements rather than being interpreted as observed walls.

Narrow passages are found by thinning connected free space to a centreline and sampling twice the clearance along its branches. A three-sample median filter reduces isolated dips. Sections narrower than 3 m qualify as narrow passages; relative bottlenecks may be wider, but must be at most 75% of the smaller shoulder width and at least 0.6 m narrower, with shoulders on

both sides within 6 m. Sections must persist for 1.2 m, and detections near junctions are excluded. Nearby candidates are merged to reduce duplicate markers.

The 19 geometry unit tests passed. In synthetic maps, the detector identified a 2.2 m corridor and a 4.2 m neck between wider rooms, each within 0.2 m of its known width. A wide room produced no passage detections, and a larger disconnected room was excluded from the widest accessible region. These checks validate controlled cases; Figure 8 still requires a cave screenshot showing that the runtime features correspond to the mapped walls.

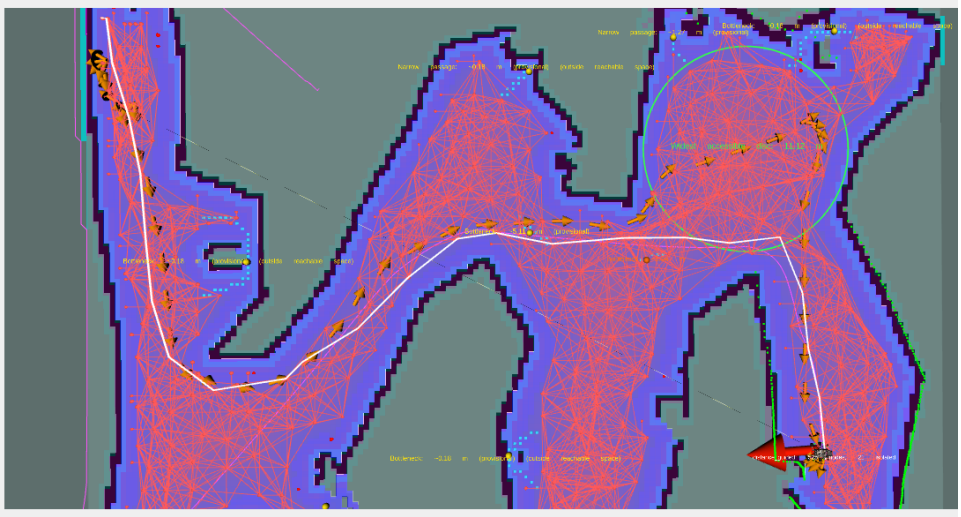


Figure 8: Cave geometry overlay showing the widest accessible disc and a narrow passage.

Only centreline sections supporting detections are displayed, preventing chamber branches from resembling planned routes. Features near unknown space are marked provisional, and inaccessible constrictions are labelled separately. Remaining limitations include coarse map resolution, underestimated widths in curved or asymmetric passages, and missed short necks near junctions. Comparing runtime widths against manual map measurements and tuning the thresholds would improve confidence in the results. These geometric estimates do not guarantee Nav2 traversability.

4.9 Advanced 4: Online Roadmap Construction

The goal of this task was to build a sparse navigation graph online while the robot explored. The initial strategy added nodes at sampled robot positions and connected nearby nodes where collision checks permitted. The improved

approaches also sample observed, reachable free space, allowing the graph to extend beyond the travelled path. A uniform sampler was compared with a distance-guided sampler that favours locally high-clearance cells. The latter applies a 70% local clearance-rank threshold to 80% of attempts, retaining 20% uniform attempts. Both favour newly observed cells on half their sampling attempts.

Nodes normally use 1.2 m spacing and a 4 m connection radius. Connections are checked against the conservative clearance map from Advanced 2, including the robot radius and safety margin. The check below also allows for the distance between samples so that a thin obstacle cannot simply fall between them.

```
def edge_valid(self, a, b):
    distance = math.dist(a, b)
    count = max(1, math.ceil(distance / (self.origin[3] * 0.5)))
    allowance = distance / count / 2
    required = self.settings.robot_radius +
        self.settings.safety_margin
    return all(self.lower_clearance(p) >= required + allowance
               for p in np.linspace(a, b, count + 1))
```

Listing 11: Collision checking along a proposed roadmap edge

Every map update revalidates existing nodes and edges. Bounded repair sampling attempts to join small components, while persistently isolated sampled nodes are retired. At capacity, a sampled node can be replaced only when its neighbours remain directly interconnected; travelled nodes are protected. RViz markers distinguish components, isolated nodes and node creation sources. The roadmap node observes exploration without issuing robot motion commands.

Table 2 reports a saved replay of seven cave map/pose snapshots captured over approximately 14 seconds. Each strategy used seed 7, capacity 120 and the same three fixed destinations, all reachable in the grid reference. The graphs started empty in an already mapped cave, so the baseline did not inherit the robot's earlier trajectory.

Measure	Baseline	Uniform	Distance-guided
Final nodes	2	106	107
Isolated nodes	2	14	2
Successful queries	0/3	0/3	1/3
Checked coverage	2.3%	7.8%	15.6%

Table 2: Short cave-snapshot replay. Coverage estimates the fraction of up to 128 sampled reachable cell centres with a collision-checked connection to the robot's graph component within 2 m.

The successful distance-guided route was 18.37 m, compared with 18.07 m for the grid reference. The other two queries failed because the graph was disconnected (Figure 9). A separate synthetic comparison over three seeds found that, at capacity 60, both sampling methods achieved 98.4% checked coverage versus 62.5% for the travelled baseline. However, at capacity 30, distance guidance solved only 7/9 reachable queries versus 8/9 for uniform sampling and 9/9 for the baseline. Distance guidance therefore did not improve every case.

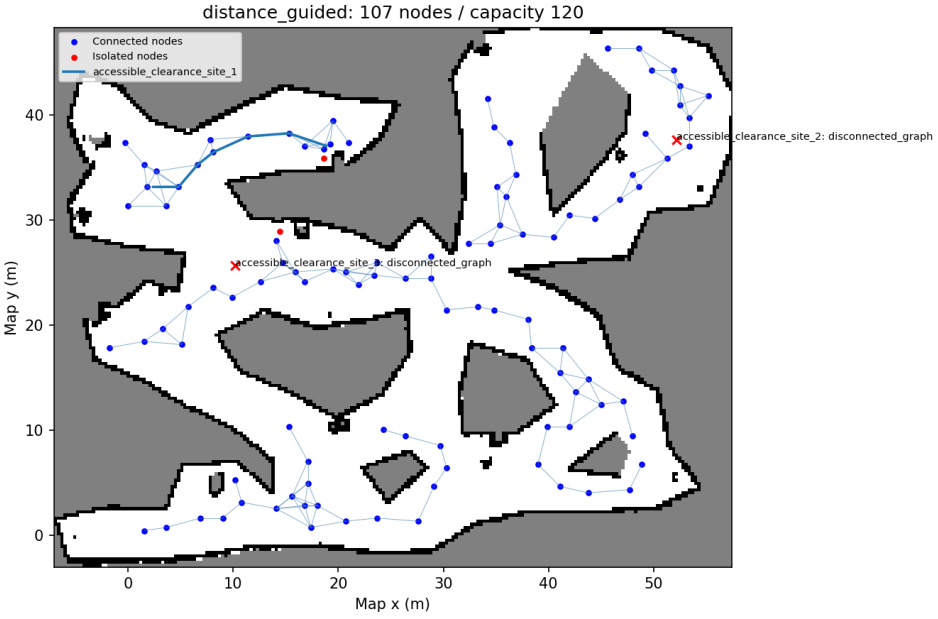


Figure 9: Distance-guided roadmap replayed from recorded cave snapshots. The thick blue route reaches one fixed destination; red crosses mark the two failed queries. Blue nodes have an edge but are not necessarily connected to the robot; red nodes are isolated. This is an offline annotated map, not an RViz screenshot or evidence of robot execution.

The main remaining difficulty is achieving connectivity within a limited node budget. These preliminary results show representation beyond the recent robot path, but not complete cave coverage. A full exploration recording, repeated seeds and capacities, and additional sampling near disconnected corridor ends would provide a stronger assessment. The grid reference shares the edge validator and is discretised, so its route length is a comparison rather than a continuous optimality or independent collision-safety guarantee.

5 Results

6 Reflection on Teamwork